

Animal Recognition System

In constrained environments using dual-camera visible/IR image capture

High-level project definition

Usage Context:

1. Heavily wooded areas
2. Low light to very-low light photography
3. Animals in motion
4. Need to identify with only partial images
5. Need to identify with non-sharp images

General process:

1. Simultaneous visible light with infrared heatmap photography
2. Expect to capture only partial images of animals
3. Preprocess and align visible & IR images
4. Manually tag images
5. Train frontend CNN image filter with visible light images to select usable frames
6. Classifier: apply selected visible-light images with matched IR to ViT model
7. Test system in offline mode

Preprocessing steps:

1. Use CNN frontend to detect any portion of animal in frame
2. Isolate animal with fixed area boarder in visible image
3. Map visible image area to lower resolution IR image
4. Tag visible-light images and IR with same ID
5. Modify ViT model to accept IR layer with RGB image data.

Models:

1. Frontend image filter: pretrained model for trail cameras MegaDetector v5
(<https://github.com/agentmorris/MegaDetector/tree/main>)
2. Classifier: Visual Transformer (ViT) pretrained model BioCLIP
(<https://github.com/Imageomics/bioclip-2>)

Hardware Considerations

1. Initial development on laptop
 2. Target deployment on Raspberry Pi 5 8GB with Hailo-8 26T AI HAT
-
-

Useful Q & A with Gemini

Prompt:

We are working on a project to identify animals in the wild, notably in dense forest areas. I am using a dual camera system of visible light and infrared image capture. I plan to train a machine learning system using tagged visible light images with parallel IR heatmap images of the same area. The animals will rarely be photographed entirely in the clear. The light levels will be relatively low even in bright noontime sun. The animals will almost always be in motion. The goal is to provide more information to train the ML with tagged visible light images with simultaneous infrared heatmaps aligned to the same image area. What is the best ML model to use, what is the best pretrained model to start from. And, please suggest any training process to optimize the accuracy of the resulting animal recognition classification system.

Gemini:

Vision Transformers (ViTs) are the best alternative to CNNs for multi-modal animal identification in dense, low-light environments.

Why Vision Transformers Superior to CNNs

- Global Context: Transformers capture long-range pixel dependencies immediately, whereas CNNs look at localized regions [1]. This helps identify partially blocked animals by linking scattered visible patches (like a tail or ear) to the full body shape [1]. [1]
 - Cross-Attention Fusion: Transformers natively fuse distinct modalities (visible light and IR heatmaps) by attending to both feature spaces simultaneously, rather than just stacking channels like a CNN [4]. [2]
 - Motion Blur Resilience: Self-attention mechanisms are historically more robust to spatial distortions and motion blur than rigid convolutional filters [5].
-

Best Pretrained Models to Start From

Model Family	Specific Recommendation	Why It Fits Your Project
Multi-Modal Transformer	ImageBind by Meta AI	It learns a joint embedding space across 6 modalities, explicitly aligning thermal (IR) and visible light images out of the box [6].
Dual-Stream Vision Transformer	Swin Transformer V2	Uses shifted windows to handle high-resolution imagery efficiently [7, 8]. V2 introduces techniques to handle low-light and resolution disparities [8, 9].
Self-Supervised Foundation	DINOv2 by Meta AI	Trained without labels to understand dense object layouts [10]. It excels at segmenting occluded objects (animals behind trees) without explicit training [10, 11].

Optimized Training Process for Maximum Accuracy

1. Implement Cross-Modal Attention Fusion

Do not simply feed a 4-channel input (RGB + IR) into the model. Instead, pass the visible light image and the IR heatmap through separate encoder branches. Use a Cross-Attention Mechanism where the IR tokens act as keys/values to query the visible light tokens [4]. This allows the heat signatures to dynamically highlight exactly where the model should look in the dark, occluded visible image [4]. [2]

2. Apply Targeted Data Augmentation

To simulate your severe environmental constraints during training, apply these specific transformations:

- Random Occlusion (Cutout/Random Erasing): Artificially block parts of the animal with black boxes or tree bark textures to force the model to rely on partial shapes [12].
- Motion Blur Filters: Apply directional Gaussian blur to training images to mimic animal movement [13].
- Low-Light / Contrast Jitter: Severely degrade the brightness and contrast of the RGB images while keeping the IR heatmap clean [14]. [3, 4]

3. Leverage Multi-Task Learning [5]

Train your model to perform two tasks simultaneously using a shared backbone: [6]

1. Primary Classification: Identifying the animal species.
2. Auxiliary Object Detection / Segmentation: Predicting a rough bounding box or mask of the animal [15]. The auxiliary task forces the backbone to learn precise spatial

boundaries of hidden animals, directly improving classification accuracy [15, 16]. [7, 8, 9, 10, 11]

4. Use Contrastive Loss (InfoNCE)

Incorporate a contrastive loss function to pull parallel visible and IR embeddings closer together in the feature space while pushing unmatched pairs apart [17]. This guarantees that even if the visible image is nearly pitch black or blurry, its feature representation aligns tightly with the highly visible thermal heatmap signature [6, 17].

Next:

- Your hardware limitations for deployment (e.g., edge devices vs. cloud servers)
- Estimated dataset size of paired images you currently possess
- Specific animal classes or diversity scale you are targeting [12]

[1] <https://www.sciencedirect.com> [2] <https://medium.com> [3] <https://medium.com> [4] <https://www.mdpi.com> [5] <https://arxiv.org> [6] <https://www.sciencedirect.com> [7] <https://www.researchgate.net> [8] <https://ieeexplore.ieee.org> [9] <https://medium.com> [10] <https://intermediait.com> [11] <https://www.sciencedirect.com> [12] <https://www.folio3.ai>

Prompt:

We would like to deploy on a NVidia Jetson but are flexible in the choice of hardware. The first goal is to overcome the partial image deficit as well as to explore any enhancement the IR image overlay can provide. Once the model is working, we will then focus on optimization for the low-power hardware for deployment.

Our initial image set is small, we are just starting the project and will only have in the hundreds of images to train. If there is a pretrained model to build on, we would like to find it.

The model will ultimately be deployed in Africa in the country of Uganda to monitor gorillas and other indigenous wild life. However, we are starting the model development and training in the United States in California. We anticipate capturing deer, mountain lions, cayotes, foxes, and occasional bears.

Gemini:

I have noted your hardware choice, dataset constraints, and regional rollout plans. Having only a few hundred images changes your immediate model architecture choice drastically. Vision Transformers trained from scratch fail on small datasets because they

lack inductive bias. Because you are starting with California wildlife (deer, coyotes, etc.) before transitioning to Ugandan species (gorillas), your best approach is a robust, lightweight foundation model designed specifically for camera traps, fine-tuned using parameter-efficient techniques.

The Best Pretrained Foundation Model to Build On

The absolute best starting point for your project is [MegaDetector \(v5 or newer\)](#).

- **Why it fits your project:** MegaDetector is a massive foundation model trained on millions of camera trap images across global ecosystems by organizations like Microsoft and Wildlife Insights.
 - **The Overlap Benefit:** It already contains extensive training data for North American wildlife (deer, coyotes, mountain lions, foxes, bears) and African wildlife (primates, leopards, elephants). It already knows what a camera trap image looks like, including low-light noise, heavy occlusion, and motion blur.
 - **Solving the Deficit:** It excels at distinguishing background forest noise from an animal body part, successfully solving your partial image deficit out of the box.
-

Step-by-Step Strategy for an NVIDIA Jetson Deployment

Phase 1: Overcoming the Small Dataset & IR Fusion (Current Focus)

With only a few hundred images, traditional heavy fusion models will overfit. Use this specific setup:##

1. **Freeze the Backbone:** Load a pretrained MegaDetector model and freeze its weights entirely.
2. **Two-Stream Extraction:** Pass your RGB image and your IR heatmap image through the identical frozen backbone as two separate streams to extract spatial feature maps.
3. **Lightweight Fusion Head:** Train a tiny, lightweight fusion head (a few attention layers or a shallow MLP) on your few hundred images. This head will learn to cross-reference the IR spatial heat map with the RGB feature map.
4. **Few-Shot Classifier:** Add a classification head on top. Since the backbone is frozen, your few hundred images are only training a small fraction of parameters, protecting you from overfitting.

Phase 2: Domain Adaptation (USA to Uganda)

Do not throw away your California model when moving to Uganda. Use Domain Adaptation:

- Keep the frozen MegaDetector backbone (which already understands both North American and African biomes).

- Swap out only the final classification head.
- When you arrive in Uganda, you will only need a few hundred images of gorillas to train the new classification head because the feature extraction layers already understand how to parse hidden objects using thermal maps.

Phase 3: Hardware Optimization for NVIDIA Jetson

Once the accuracy is stable, you can easily optimize this workflow for low-power edge hardware:

- NVIDIA TensorRT: Convert your final PyTorch/ONNX model into a TensorRT engine directly on the Jetson. This fuses layers and quantizes weights to maximize FPS. [1, 2, 3, 4, 5]
- INT8 Quantization: Quantize your model from FP32 to INT8 precision. This reduces the memory footprint and significantly speeds up processing on the Jetson's GPU/DLA cores with negligible drops in accuracy. [6]

Next:

- The specific Jetson model you plan to test on (e.g., Nano, Xavier, Orin Nano, or Orin NX)
- The format of your IR data (e.g., raw temperature arrays or pre-rendered 3-channel heatmaps)
- Your frame rate requirements for the edge system (e.g., real-time processing vs. batch processing every few seconds) [7, 8, 9]

[1] <https://github.com> [2] <https://thinkrobotics.com> [3] <https://forums.developer.nvidia.com> [4] <https://www.mdpi.com> [5] <https://pmc.ncbi.nlm.nih.gov> [6] <https://wizzdev.com> [7] <https://www.pipeless.ai> [8] <https://blog.roboflow.com> [9] <https://forums.developer.nvidia.com>

Prompt:

We are not committed to the Nvidia Jetson, is there a better hardware recommendation for using MegaDetector?

Gemini:

You do not need to restrict yourself to an NVIDIA Jetson. MegaDetector does not strictly require an NVIDIA GPU to run. [1] The official May 2026 release of MegaDetector V6 explicitly includes compact, optimized variants optimized for low-power edge environments. Because the underlying architecture of MegaDetector is built on the

highly versatile YOLO framework, you have excellent hardware alternatives that offer significant advantages for a remote deployment in Uganda. [1, 2, 3]

Top Hardware Alternatives to the NVIDIA Jetson

For field deployment, you should choose a hardware platform based on whether you are doing Batch Off-Grid Processing (collecting SD cards and running AI locally in a basecamp) or Real-Time Remote Edge Processing (the AI runs directly inside the camera box in the forest).

Hardware Platform	Best For	AI Performance	Power Consumption	Why It's Better Than Jetson
Mini PC / Used Laptop (Intel/AMD CPU)	Batch Basecamp Processing	~2–5 images/sec (V6 Compact)	15W – 45W	Easiest Software Stack. MegaDetector runs via a simple 3-line Python script or GUI without compiling specific hardware drivers. Perfect for processing thousands of images from SD cards at a local camp.
Raspberry Pi 5 + Hailo-8L AI Kit	Real-Time Active Camera Trap	13 TOPS (~40 FPS for YOLO models)	5W – 7W	Best Performance-per-Dollar & Power. Highly energy-efficient. You can easily compile a YOLO/MegaDetector model into a Hailo .hef file. It consumes significantly less idle power than a Jetson, extending solar/battery life in the jungle.
Raspberry Pi 4/5 + Google Coral USB	Low-Budget Edge Inference	4 TOPS	2W – 4W	Ultra-Low Idle Power. The Google Coral Edge TPU runs quantized TensorFlow Lite models perfectly. It has a very low power draw, though it requires strict model quantization (INT8) before deployment.

Why Skipping Jetson Might Be Smarter for Your Project

While the NVIDIA Jetson series (especially the newer Orin Nano and Orin NX) is incredibly powerful, it brings two distinct bottlenecks to a conservation project: [4, 5]

1. The Software "Lock-In" Support Overhead: The Jetson requires navigating NVIDIA's complex JetPack Linux environments and compiling specific PyTorch-to-TensorRT architectures. In a remote setting like Uganda, a standard CPU stack (like an x86

Mini PC) or an active community stack (like the Raspberry Pi) is vastly easier for field researchers to debug, reinstall, or swap out. [5]

2. Power Budget Dominance: Wildlife camera traps spend 99% of their time waiting for an animal to cross a passive infrared (PIR) sensor trigger. While the Jetson is blazing fast during active inference, its idle power consumption is higher than a microcontroller or a sleep-cycled Raspberry Pi system.

Recommendation for Your Roadmap

- Right Now (California Phase): Develop, align, and train your dual-stream visible/IR fusion layers on a standard PC or laptop with a GPU. Do not waste time debugging edge hardware constraints until your model is successfully fusing the images and detecting partially blocked animals. [1]
- Deployment Phase (Uganda Phase): If your camera setup triggers on a motion sensor and processes single images frame-by-frame, deploy using a Raspberry Pi 5 paired with a Hailo-8L accelerator module. It gives you the best balance of field durability, low power draw, and high-speed local processing. [6, 7, 8]

Next:

- The field power strategy you intend to use in Uganda (e.g., solar panels, external lithium battery packs, or standard AA cells)
- The physical enclosure size limitations for the camera system
- The connection type of your visible/IR dual cameras (e.g., USB, MIPI CSI ribbons, or network RTSP)

[1] <https://microsoft.github.io> [2] <https://wildlabs.net> [3] <https://thefieldco.com> [4] <https://www.reddit.com> [5] <https://www.reddit.com> [6] <https://www.jeffgeerling.com> [7] <https://industrialmonitordirect.com> [8] <https://www.youtube.com>

Prompt:

What model does MegaDetector utilize?

Gemini:

The Core Architecture of MegaDetector

MegaDetector is built entirely on CNN-based object detection frameworks—specifically from the YOLO (You Only Look Once) family: [2, 3] It does not natively run a Vision Transformer (ViT) model. [1]

- MegaDetector v5 utilizes a YOLOv5 architecture.
- MegaDetector v6 (released by the Microsoft AI for Good Lab) expands into multiple selectable CNN configurations including YOLOv9, YOLOv10, and RT-DETR. [3, 4]

While RT-DETR (Real-Time DEtection TRansformer) introduces transformer-style multi-scale feature interactions inside its internal processing neck, it still relies on a classic CNN backbone (such as ResNet or HGNet) to extract the primary visual features. MegaDetector as a whole does not use the patch-based, self-attention-heavy visual processing characteristic of pure Vision Transformers (like ViT-Base or Swin). [1, 4, 5]

How Vision Transformers Are Actually Used with MegaDetector

In standard conservation workflows, researchers do not use MegaDetector as a transformer. Instead, they use it in a two-stage hybrid pipeline where MegaDetector and Vision Transformers handle different tasks: [1, 6]

1. Stage 1 (The CNN/YOLO Detector): MegaDetector filters out empty "blank" background images and generates precise bounding box crops around any localized animals, people, or vehicles. [7]
2. Stage 2 (The Downstream Vision Transformer Classifier): Biologists take those cropped image patches and feed them into specialized downstream wildlife classification models—such as DeepFaune (a self-supervised ViT framework), BioCLIP, or DINOv2—to handle the actual species identification. [1, 8, 9, 10]

This hybrid separation allows the CNN architecture to efficiently handle raw object location tasks on the edge while leaving complex species distinction to a transformer model. [6, 11]

Questions:

- Do you want to use MegaDetector as a pre-processor for a downstream classification layer
- Do you would prefer a pure Vision Transformer alternative designed for camera traps
- Which specific YOLO/MDv6 architecture variation you're leaning toward for hardware testing

[1] <https://onlinelibrary.wiley.com> [2] <https://megadetector.readthedocs.io> [3] <https://github.com> [4] <https://github.com> [5] <https://ieeexplore.ieee.org> [6] <https://www.researchgate.net> [7] <https://microsoft.github.io> [8] <https://onlinelibrary.wiley.com> [9] <https://arxiv.org> [10] <https://github.com> [11] <https://papers.ssrn.com>

Prompt:

I assume that the pretrained model of MegaDetector outweighs the advantages of the more advanced Vision Transformer?

Gemini:

Yes, given your exact project parameters, the pretraining of MegaDetector firmly outweighs the architectural advantages of a Vision Transformer (ViT). While a Vision Transformer is theoretically better at capturing global context, a model's architecture is only as good as the data that fills it. In your current phase, MegaDetector's massive pretraining data provides a critical advantage over an advanced ViT.

Why MegaDetector Overrules a ViT for Your Setup

1. The "Small Data" Wall

Vision Transformers have no inherent inductive biases (like the assumption that pixels close to each other are related). Because of this, ViTs require hundreds of thousands of images just to learn how to identify basic edges and shapes. If you attempt to train or heavily fine-tune a pure ViT with only a few hundred images, the model will overfit immediately, memorizing background elements like specific tree branches rather than the animals. MegaDetector has already processed millions of camera trap images. It already possesses the deep visual "common sense" required to handle low-light pixel noise, motion blur, and structural patterns of wildlife out of the box.

2. Resolving the Partial Image Deficit

A core strength of MegaDetector's extensive training is its ability to identify occluded subjects. Because it has seen thousands of examples of deer hidden behind thick brush or a mountain lion obscured by leaves, its frozen feature extraction layers are highly resilient to partial views. You don't need a transformer's global attention mechanism to link a hidden tail to a head if the CNN backbone already recognizes a fractional texture as an animal boundary.

3. Cross-Modal Flexibility

Because you are building a custom dual-camera setup (visible + IR), you will have to train custom fusion layers. MegaDetector allows you to extract robust, generalized feature maps from your low-light visible images instantly. You can then train a tiny, lightweight multi-modal coupling head using your few hundred custom paired images without altering the underlying foundation model.

The Verdict for Your Roadmap

Choosing an advanced ViT right now would force you to solve two complex problems at once: compensating for a severe lack of data while simultaneously trying to optimize a data-hungry architecture. By building on top of MegaDetector's massive pretrained base, you neutralize the data scarcity issue immediately, allowing your team to focus entirely on perfecting the alignment and fusion of your infrared heatmap overlays.

Next:

- The structural arrangement of your dual-camera rig (e.g., side-by-side or using a beam splitter for exact pixel alignment)
 - The programming language or framework you prefer for building the custom IR fusion layer (e.g., PyTorch, TensorFlow)
 - Your primary target metric for this phase (e.g., maximum classification accuracy vs. reducing false-trigger background blanks)
-
-

Prompt:

Using MegaDetector as a pre-processor for a downstream classification layer seems like a perfect frontend. What ViT would you recommend to follow MegaDetector?

Gemini:

Pairing MegaDetector as a bounding-box frontend with a downstream Vision Transformer (ViT) classifier is the exact gold standard utilized by modern ecological AI frameworks. MegaDetector isolates the cropped patch of the animal (filtering out the dense forest background noise), and the ViT evaluates only that focused patch. [1, 2, 3, 4] Because you are starting with a small data pool (hundreds of images) and planning a cross-continental model migration (California to Uganda), you should choose from three specific, highly optimized ViT models:

1. BioCLIP (Top Recommendation)

Developed specifically as a foundation model for the "Tree of Life," [BioCLIP](#) is an open-source vision-language model trained on the massive iNaturalist dataset. [5]

- Why it fits your project: It is structured using a ViT-Base backbone but leverages contrastive language-image pretraining. It already understands fine-grained biological traits across a massive taxonomy, including both North American mammals and African primates. [5]

- The Few-Shot Advantage: Because it has a deep intrinsic understanding of animal anatomy, you can freeze the backbone and train a tiny classification layer on your few hundred California images. When you move to Uganda, you can use its zero-shot capabilities or a tiny handful of gorilla crops to get high accuracy instantly. [5]

2. DeepFaune (Best Camera-Trap Native ViT)

[DeepFaune](#) is a self-supervised Vision Transformer specifically pre-trained entirely on millions of trail camera image crops. [2, 3]

- Why it fits your project: Traditional ViTs are trained on pristine, well-lit photos (like ImageNet). DeepFaune was trained on raw, imperfect, low-light, blurry camera-trap crops. Regional variants like DeepFaune New England (DFNE) achieve over 97% accuracy on North American wildlife by simply swapping the final classification head over the frozen backbone. [3, 4, 6, 7]
- The Fusion Benefit: Since it already expects degraded, low-contrast bounding boxes, it is the perfect base for appending your custom IR heatmap fusion layers without causing the network to break down.

3. DINOv2 by Meta AI (Best for Extreme Occlusion)

If your dense forest environment results in animals that are heavily blocked by branches or brush, [DINOv2 \(ViT-scaled variants\)](#) is an incredible self-supervised foundation model.

- Why it fits your project: DINOv2 was trained using self-distillation to understand dense, pixel-level object boundaries without relying on text labels. [8]
- The Occlusion Master: DINOv2's internal self-attention maps automatically learn structural segmentation. It is uniquely gifted at grouping sparse fragments of an animal (e.g., seeing a patch of fur through thick leaves and recognizing it as part of a single continuous hidden object). [8]

How to Structure Your Pipeline Code

Your software pipeline will flow natively in PyTorch like this:

1. The Crop Phase: Send your raw visible/IR synchronized frames through MegaDetector. MegaDetector returns a bounding box coordinate [x_min, y_min, x_max, y_max]. [1, 2]
2. The Context Expansion: Crop both the visible frame and the IR heatmap using those coordinates, but expand the box dimensions by 30% to 50%. This guarantees your ViT receives a tiny bit of surrounding context, which helps the self-attention layers compute local-vs-global contrast in low-light. [1, 8]

3. The Multi-Modal Attention Head: Flatten the visible crop patches and the IR crop patches. Instead of standard classification, use a shallow 2-layer Transformer Encoder where the IR patches act as the Key and Value, allowing the heat signature to guide the Query of the dark visible frame. [9, 10]
-

Next:

- Which model style aligns best with your development timeline (biological language-image matching like BioCLIP vs. raw pixel boundary tracking like DINOv2)
- The dimension size your camera pipeline outputs (e.g., 1080p, 4K, or square aspect ratios)
- If you want a basic PyTorch code boilerplate showing how to load MegaDetector crops into a frozen ViT backbone [10]

[1] <https://arxiv.org> [2] <https://ceur-ws.org> [3] <https://pmc.ncbi.nlm.nih.gov> [4] <https://www.usgs.gov> [5] <https://www.youtube.com> [6] <https://www.sciencedirect.com> [7] <https://besjournals.onlinelibrary.wiley.com> [8] <https://www.lightly.ai> [9] <https://builder.aws.com> [10] <https://towardsdatascience.com>

Prompt:

Can any of the above ViT models take advantage of the simultaneous IR heatmap images that will accompany our visual images?

Gemini:

No, none of these pretrained ViT models can take advantage of your IR heatmap images out of the box.

BioCLIP, DeepFaune, and DINOv2 are strictly single-modality RGB models. Their architecture requires exactly a 3-channel input (R, G, B). If you feed a 4th thermal channel or an independent IR image into them directly, the mathematical dimensions will mismatch, and the code will crash.

However, you can absolutely modify the pipeline to force these models to leverage your dual-camera imagery. Because you are using a transformer, you have unique ways to inject the IR data.

How to Modify the ViTs to Accept Your IR Overlay

Strategy A: Early Fusion (Input Adaptation)

This is the simplest method. You modify the very first layer of the ViT while keeping the rest of the model frozen.

- How it works: You expand the first weight matrix (the "Patch Embedding" layer) from 3 input channels to 4 channels ($R, G, B, +IR$ Heatmap).
- The Training Requirement: You randomly initialize the weights for that 4th channel. You will use your few hundred paired images to train only this input patch embedding layer, keeping the million-image backbone frozen.

Strategy B: Late Fusion (Dual-Stream Cross-Attention)

This is the most powerful method for transformers. You use two completely separate instances of the model.

- How it works: You pass your RGB crop through a frozen instance of the ViT to get "RGB tokens." Simultaneously, you duplicate your 1-channel IR heatmap into a 3-channel grayscale image and pass it through an identical frozen instance of the ViT to get "Thermal tokens."
- The Fusion Bridge: You place a tiny, un-frozen Cross-Attention layer right after the backbones. This layer maps the thermal tokens to the RGB tokens, dynamically using the heat signatures to tell the model exactly which blurry, low-light RGB patches contain an actual animal body part.

Which Model Handles These Modifications Best?

Model	Multi-Modal Adaptability Rating	Integration Blueprint
DINOv2	★★★★★ (Best Choice)	DINOv2's self-supervised tokens are highly dense and spatial. It maps pixel boundaries perfectly. Using Late Fusion (Strategy B) with DINOv2 allows your thermal tokens to directly mask and highlight the hidden physical boundaries in the dark RGB image.
BioCLIP	★★★★☆	BioCLIP is trained on text and image pairs. If you use Early Fusion (Strategy A), you can pass the fused 4-channel image into the vision backbone. The model will still be able to map those features to its existing text vocabulary (e.g., "Gorilla" or "Coyote") with very little training data.
DeepFaune	★★★☆☆	DeepFaune is pre-trained exclusively on low-quality camera trap images. While highly resilient to bad lighting, its internal weights are tightly locked to traditional RGB patterns. Modifying its input layers requires very careful regularization to avoid losing its native trail-camera expertise.